基于大语言模型增强的序列推荐系统：从 GPT-2 特征融合到 miniHLLM

小组成员：（填写）

摘要

本项目在 MovieLens-1M 上设计并实现了两代大语言模型（LLM）增强的推荐系统。第一阶段完成课程基础框架：用冻结的 GPT-2 提取电影标题与类型的文本特征，与用户、电影 ID 向量拼接后由神经协同过滤（NCF）打分，BPR 损失训练。第二阶段是本项目的核心设计 miniHLLM：参照 ByteDance 的 HLLM 分层结构，把 GPT-2 同时用作物品编码器和用户序列模型，模型中不含任何 ID embedding，物品的全部表示来自文本，用户序列由预训练 GPT-2 以向量序列方式读入并微调，对全部候选电影做 softmax 排序学习。在按时间 leave-one-out 的全量排序协议下，miniHLLM 测试 NDCG@10 达到 0.173 ± 0.001，优于收敛后的 SASRec（0.164）和加入文本特征的 SASRec（0.170），且 25 轮即收敛，三种子方差为所有模型最小。消融实验表明：语言预训练带来 0.6 个点的稳定增益；加回 ID embedding 对 miniHLLM 无帮助；全参数微调的 GPT-2 把容量集中在热门电影上，冻结 GPT-2 只训练 1.8M 参数的适配器反而在冷门电影上表现最好。报告最后讨论了方案的局限与改进方向。

1 项目介绍

1.1 项目背景

协同过滤是推荐系统的主流范式：把每个用户和物品映射为可训练的 ID 向量，从历史交互中学习二者的匹配关系。这一范式有两个固有缺陷。其一是稀疏性，交互次数少的物品，其 ID 向量得不到充分训练；其二是冷启动，新物品没有任何交互，ID 向量完全是随机初始化，无法被推荐。物品本身的内容信息（标题、类型、描述）不依赖交互，能够缓解这两个问题，但传统做法需要人工设计内容特征。

大语言模型为内容信息的利用提供了新途径。预训练语言模型把任意文本映射为携带语义的稠密向量，“Toy Story” 和 “A Bug’s Life” 的向量距离天然很近，无需任何交互数据。当前 LLM 增强推荐的研究大致沿两条路线展开：

- **LLM 作为特征编码器**：用冻结的语言模型为物品生成文本向量，作为协同过滤模型的附加输入。实现简单，但语言模型本身不参与推荐任务的学习。
- **LLM 作为推荐骨干**：直接用语言模型处理用户的行为序列。代表工作 HLLM（Chen et al., 2024）提出分层结构：Item LLM 把物品文本压缩为一个向量，User LLM 把用户历史的物品向量序列当作 “token 序列” 读入，预测下一个物品的向量，两个模型端到端训练，整个系统不含 ID embedding。

本项目沿这两条路线各实现一个系统，并通过统一的对照实验回答它们的差异。

1.2 研究目标

围绕 “LLM 到底在推荐中起了什么作用”，本项目设置三个具体问题：

- **Q1**：仅凭 LLM 的文本表示、不使用任何 ID embedding，推荐效果能否达到或超过基于 ID 的序列推荐模型？
- **Q2**：语言模型在文本上的预训练知识，能否迁移到用户行为序列的建模上？也就是说，预训练的 GPT-2 作为用户序列模型，是否比同结构随机初始化的 Transformer 更好？
- **Q3**：LLM 文本表示对冷门物品的推荐有多大帮助，这种帮助与模型微调程度的关系是什么？

1.3 整体方案

项目分两个阶段。阶段一的代码位于 `recommender/base_LLM_NCF/`，阶段二位于 `recommender/miniHLLM/`，两者相互独立，只共享 MovieLens-1M 数据与本地 GPT-2 权重。

阶段一：GPT-2 特征增强的 NCF。完成课程基础框架的四处核心实现：GPT-2 隐藏状态的掩码平均池化、排除用户历史的负采样、用户 ID 向量、电影 ID 向量与电影文本向量的融合打分、BPR 损失。该阶段验证了 LLM 文本特征可以嵌入经典协同过滤模型，但课程框架采用记录级随机划分，没有基线对照，无法回答上述三个问题。

阶段二：miniHLLM 序列推荐。把推荐任务改为更贴近真实场景的“预测下一部电影”，按时间 leave-one-out 划分，设计 miniHLLM 并构造一组严格受控的对照模型：同一物品表示下的 SASRec 骨干、同一骨干下的随机初始化、冻结与微调、加与不加 ID。所有模型使用相同的数据划分、评测口径、训练预算和三个随机种子，附加按物品热度分桶的冷启动评测。

2 方法设计

2.1 阶段一：GPT-2 特征增强的 NCF

文本特征提取。每部电影构造文本 `Title: < 标题 >. Genres: < 类型 >.`，送入冻结的 GPT-2 small（12 层，隐藏维度 768），取最后一层隐藏状态 $\mathbf{H} \in \mathbb{R}^{L\times 768}$，按 attention mask m 做掩码平均池化：

$$\mathbf{t}_i = \frac{\sum_{l=1}^{L} m_l \, \mathbf{h}_l}{\sum_{l=1}^{L} m_l}$$

3,883 部电影的特征按原始电影 ID 存入形状 $(3953, 768)$ 的表，ID 空洞对应零向量。

融合打分。用户 u 与电影 i 的分数由三个 32 维向量拼接后经 MLP 得到：

$$s(u,i) = \mathrm{MLP}\big([\mathbf{p}_u;\ \mathbf{q}_i;\ \mathbf{W}\mathbf{t}_i]\big)$$

其中 $\mathbf{p}_u$、$\mathbf{q}_i$ 为可训练 ID 向量，$\mathbf{W} \in \mathbb{R}^{32\times 768}$ 为可训练投影，MLP 为 Linear(96, 64)–ReLU–Linear(64, 1)。GPT-2 特征以 buffer 形式固定。

BPR 损失。每条正样本 (u, i^+) 配一部用户训练历史之外的随机电影 i^-：

$$\mathcal{L}_{\mathrm{BPR}} = -\frac{1}{B}\sum_{b=1}^{B} \log \sigma\big(s(u_b, i_b^+) - s(u_b, i_b^-)\big)$$

该阶段 LLM 的作用是**物品侧的冻结语义编码器**，不参与训练。

2.2 阶段二：miniHLLM

2.2.1 问题定义与数据流

每个用户的评分按时间戳排序得到序列 $[i_1, i_2, \ldots, i_n]$。采用 leave-one-out 划分：i_n 为测试目标，i_{n-1} 为验证目标，$[i_1, \ldots, i_{n-2}]$ 用于训练。与阶段一 “预测用户是否会看某部电影” 不同，这里的任务是 “给定到目前为止的观看历史，预测下一部”，是标准的序列推荐设定（Kang & McAuley, 2018）。

训练时采用**逐位置监督**：输入 $[i_1, \ldots, i_{n-3}]$，目标 $[i_2, \ldots, i_{n-2}]$，序列中每个位置 t 都要预测 i_{t+1}。一个用户贡献最多 L 个训练信号（L 为截断长度 100），6,040 位用户每轮约 29 万个预测任务，远多于 “每个用户只预测最后一部” 的做法。

进入模型的张量约定如下（B 为 batch 中的用户数，$N = 3953$ 为电影编号上限加一）：

张量	形状	含义
`seq`	$[B, L]$	电影编号，最近 L 部，不足右侧补 0；0 不是任何电影
`target`	$[B, L]$	每个位置的下一部电影编号，补位填 -100
`pad_mask = seq > 0`	$[B, L]$	真实位置为 True
`valid_item_mask`	$[N]$	3,883 部有效电影为 True，编号空洞与 0 号为 False

评测时输入完整历史，只取最后一个真实位置的输出对全部 3,883 部电影排序。

2.2.2 总体结构

```
电影文本 "Title: Toy Story (1995). Genres: Animation, Children's, Comedy."
  |
  v  Item LLM：冻结 GPT-2，掩码平均池化 -> t_i (768 维，离线计算一次)
  |
  v  逐维标准化 -> 适配器 Linear-GELU-Linear-LayerNorm -> e_i (768 维)
  |                                  整张物品表 E: [3953, 768]
  v
用户历史 seq [B, L] -> 查表 E[seq] -> 向量序列 [B, L, 768]
  |
  v  User LLM：GPT-2 以 inputs_embeds 读入向量序列，因果注意力，全参数微调
  |
  v  位置输出 h_t (768 维)，经 out_proj 回到物品空间
  |
  v  s_{t,i} = cos(h_t, e_i) / tau，对全部电影 softmax 交叉熵，目标 i_{t+1}
```

一次前向的形状变化：

步骤	代码	输出形状
计算整张物品表	`table = item_encoder()`	$[N, 768]$
查表	`x = table[seq]`	$[B, L, 768]$
用户序列模型	`h = backbone(x, pad_mask)`	$[B, L, 768]$
回到物品空间	`h = out_proj(h)`	$[B, L, 768]$
对全部电影打分	`logits = cos(h, table) / tau`	$[B, L, N]$
交叉熵	`cross_entropy(logits, target)`	标量

模型中可训练的部分只有四块：适配器（1.2M）、User LLM（85M，可冻结）、`out_proj`（0.6M）和温度 τ（1 个标量）。模型里**没有 ID embedding**，物品的全部表示来自文本，这是 HLLM 区别于传统序列推荐的根本点，也是 Q1 要检验的设计。

2.2.3 物品侧：Item LLM 与物品表

文本编码。物品侧沿用阶段一的做法：每部电影构造文本 `Title: <` 标题 `>. Genres: <` 类型 `>.`，由冻结的 GPT-2 small 编码，取最后一层隐藏状态做掩码平均池化得到 $\mathbf{t}_i \in \mathbb{R}^{768}$。3,883 部电影离线计算一次，训练中不再调用 GPT-2 的文本前向。HLLM 原文在文本末尾附加一个 `[ITEM]` token、取其隐藏状态作为物品向量，本项目同时保存了平均池化与末 token 两种读出，正式实验使用平均池化。

逐维标准化。GPT-2 的隐藏状态具有强各向异性（Ethayarajh, 2019）：少数“流氓维度”（Timkey & van Schijndel, 2021）的数值是其余维度的几十倍，且所有电影在这些维度上取值相近，导致任意两部电影的余弦相似度都在 0.9 以上，文本向量之间几乎没有区分度。为此在 3,883 部有效电影上做逐维标准化：

$$\tilde{\mathbf{t}}_i = \frac{\mathbf{t}_i - \boldsymbol{\mu}}{\boldsymbol{\sigma}},$$

$$\boldsymbol{\mu} = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \mathbf{t}_i, \qquad \boldsymbol{\sigma}^2 = \frac{1}{|\mathcal{I}|-1}\sum_{i\in\mathcal{I}} (\mathbf{t}_i - \boldsymbol{\mu})^2$$

统计量在模型初始化时计算一次，以 buffer 形式保存。预实验中，不做标准化的 GPT-2 版本前 10 轮验证 HR@10 停在 0.04 到 0.06，标准化后第 2 轮即达 0.127。

适配器。标准化后的向量经一个可训练的两层 MLP 得到物品向量：

```
self.adapter = nn.Sequential(
    nn.Linear(768, 768), nn.GELU(),
    nn.Linear(768, 768),
    nn.LayerNorm(768),
)
```

适配器承担两个职责。其一是**语义空间到推荐空间的映射**：语言模型认为 *Toy Story* 与 *Toy Story 2* 几乎相同，推荐任务可能需要区分它们；语言模型不知道 *Pocahontas* 与 *Mulan* 在观众群上的关联，推荐任务需要建立它。其二是替代 HLLM 中对 Item LLM 的端到端微调：受单卡显存限制，本项目冻结 GPT-2，只训练其输出之上的 1.2M 参数。末尾的 LayerNorm 把每个物品向量的尺度拉齐，使后续的余弦打分和 User LLM 的输入分布稳定。

物品表 E 的计算方式。`ItemEncoder.forward()` 不接受任何输入，每次前向返回整张表 $\mathbf{E} \in \mathbb{R}^{N \times 768}$：

```
def forward(self):
    table = self.adapter(self.text_features)      # [N, 768]，每步重新计算
    if self.use_id:
        table = table + self.id_embedding.weight  # 可选的 ID 分支（H4）
    return table
```

传统推荐模型只为 batch 中出现的物品计算向量，这里每步重算整张表，原因有二：一是打分时需要和全部电影比较，整张表本来就要算；二是 3953×768 的计算量可以忽略。更重要的是**输入侧查表与输出侧打分共用这一张表**：用户历史中的电影用 $\mathbf{e}_i$ 表示，候选电影也用 $\mathbf{e}_i$ 打分。这保证了“用户看过 A，模型推荐与 A 语义相近的 B”这一行为有明确的几何含义，也是冷启动能力的来源：一部没有任何交互的新电影，只要有标题和类型，就能得到一个与同类电影相近的 $\mathbf{e}_i$，从而被打分和推荐。

可选的 ID 分支是 `nn.Embedding(N, 768)`，与文本向量相加，等价于给每部电影一个可学习的偏移。该分支仅在对照模型 H4 中开启。

2.2.4 用户侧：User LLM

把电影当作词，把观看历史当作句子。GPT-2 的常规用法是输入 token 编号，内部先查词表得到 $[B, L, 768]$ 的词向量，加位置编码后进入 12 层因果 Transformer。Hugging Face 实现提供 `inputs_embeds` 参数，允许跳过查词表、直接提供 $[B, L, 768]$ 的向量序列。User LLM 的实现只有一行：

```
class GPT2Backbone(nn.Module):
    def forward(self, x, pad_mask):          # x: [B, L, 768]
        return self.gpt2(inputs_embeds=x,
                         attention_mask=pad_mask.long()).last_hidden_state
```

用户历史对应的物品向量序列 $[\mathbf{e}_{i_1}, \ldots, \mathbf{e}_{i_L}]$ 就是 `x`。GPT-2 自带的位置编码 `wpe`、因果注意力掩码和 12 层结构原样保留，预训练权重全部载入并参与微调。`attention_mask` 屏蔽右侧补位。词表 `wte` 在这一路径上不会被使用，实现中将其替换为 `nn.Identity()` 以节省显存。

因果性。位置 t 的输出 $\mathbf{h}_t$ 只能依赖 $\mathbf{e}_{i_1}, \ldots, \mathbf{e}_{i_t}$，否则训练时模型可以直接“看到”目标 i_{t+1}。GPT-2 作为自回归语言模型天然满足这一约束。对照用的 SASRec 骨干需要显式构造上三角 $-\infty$ 掩码：

```
causal = nn.Transformer.generate_square_subsequent_mask(L)
h = self.encoder(x + self.pos_embedding(pos), mask=causal,
                 src_key_padding_mask=~pad_mask)
```

实现过程中用“改动未来位置的输入、检查过去位置的输出是否不变”验证了两个骨干的因果性。

位置编码。 GPT-2 的 `wpe` 是在文本上预训练的绝对位置编码（最长 1,024），序列推荐的位置 t 与文本的位置 t 并无语义对应，本项目直接复用并随其余参数一起微调，未单独测试重新初始化的效果。SASRec 骨干使用可学习的位置编码。

三**种变体**。通过两个开关得到 Q2、Q3 所需的对照：

变体	构造	可训练参数
预训练并微调（H1，主模型）	`GPT2Model.from_pretrained(path)`	87.6M
随机初始化（H2）	用同一份 `GPT2Config` 构造 `GPT2Model`，结构相同、权重随机	87.6M
冻结（H3）	预训练权重，`requires_grad_(False)`，优化器中过滤	1.8M

三者的输入、输出接口和其余所有部件完全相同，差异被严格限制在 User LLM 的权重上。

投影层。 `in_proj` 把物品向量投到骨干隐藏维度，`out_proj` 把骨干输出投回物品空间。GPT-2 骨干两侧均为 768，`in_proj` 退化为恒等映射；SASRec 骨干维度为 64，两个投影层负责维度转换。

2.2.5 打分函数与训练目标

余弦打分与温度。位置 t 的输出 $\mathbf{h}_t$ 对电影 i 的分数为余弦相似度除以温度：

$$s_{t,i} = \frac{1}{\tau} \cdot \frac{\mathbf{h}_t^\top \mathbf{e}_i}{\|\mathbf{h}_t\| \, \|\mathbf{e}_i\|}$$

```
tau = self.log_tau.clamp(math.log(0.01), math.log(1.0)).exp()
logits = F.normalize(h, dim=-1) @ F.normalize(table, dim=-1).t() / tau
logits = logits.masked_fill(~self.valid_item_mask, float("-inf"))
```

余弦把分数限制在 $[-1, 1]$，温度再放大到 softmax 可以分辨的尺度；$\tau = 0.05$ 时 logits 范围为 ± 20。τ 以对数形式存为可学习参数，保证恒正，并限制在 $[0.01, 1]$ 内。各模型训练结束时 τ 落在 0.024 到 0.044 之间，均低于初始值，说明模型倾向于更尖锐的分布。选择余弦而非无界的点积，一是与 HLLM 的 InfoNCE 形式一致，二是归一化使 User LLM 输出尺度的变化不会直接改变 logits 的量级，训练更稳定。编号空洞与 0 号的分数置为 $-\infty$，softmax 后概率严格为 0。

全量 softmax 交叉熵。训练目标是在全部有效电影上的多分类交叉熵：

$$\mathcal{L} = -\frac{1}{|\mathcal{T}|} \sum_{(t,\, i^+) \in \mathcal{T}} \log \frac{\exp(s_{t,i^+})}{\sum_{j \in \mathcal{I}} \exp(s_{t,j})}$$

$\mathcal{T}$ 为 batch 中所有非填充位置，$\mathcal{I}$ 为 3,883 部有效电影。实现上把 $[B, L, N]$ 的 logits 摊平成 $[B{\cdot}L, N]$，每行是一个 3,883 类的分类问题，`ignore_index=-100` 跳过补位：

```
F.cross_entropy(logits.reshape(-1, N), target.reshape(-1),
                ignore_index=-100)
```

与阶段一的 BPR 和 HLLM 的采样 InfoNCE 相比：

目标	每个正样本比较的负样本	梯度信号
BPR（阶段一）	1 个随机负样本	只推开一对
采样 InfoNCE（HLLM）	batch 内 + 采样的数百个	推开一批
全量 softmax（本项目）	其余全部 3,882 部	推开所有

HLLM 的候选集是亿级，只能采样；本项目候选仅 3,883 部，全量 softmax 是 InfoNCE 在负样本取全集时的精确形式，无采样偏差，每个位置同时把目标推高、把其余所有电影推低。这是 miniHLLM 25 轮即收敛的主要原因之一。

2.2.6 训练与推理流程

训练循环。每轮按用户随机打乱，batch 为 128 个用户（即 128×100 个位置）：

```
with torch.autocast("cuda", dtype=torch.bfloat16):      # 矩阵乘用 bf16
    loss = model.loss(seq, tgt)
optimizer.zero_grad(set_to_none=True)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # 梯度范数裁剪
optimizer.step()                                          # AdamW
```

打分矩阵的计算在 `autocast` 内显式切回 fp32（见 2.4 节）。每轮结束在验证集上计算 NDCG@10，刷新最佳时保存权重副本，连续 20 轮未刷新则停止，回滚到最佳权重后再评测测试集。测试集不参与任何决策。

推理。评测时输入完整历史，只取每条序列最后一个真实位置的输出：

```
last = (seq > 0).sum(1) - 1                # 每个用户的真实长度减一
h_last = h[torch.arange(B), last]          # [B, 768]
scores = self.scores(h_last, table)        # [B, N]
```

再屏蔽该用户历史中出现过的电影（置 $-\infty$），对剩余电影排序。生成 Top-K 推荐只需取分数最高的 K 部。

计算与显存。一步训练的主要开销是 GPT-2 处理 $128 \times 100 = 12{,}800$ 个位置（约 $2 \times 85\text{M} \times 12{,}800 \approx 2.2$ TFLOPs 前向），以及打分矩阵乘 $[12{,}800, 768] \times [768, 3{,}953]$；logits 张量 $[128, 100, 3953]$ 在 fp32 下占 200 MB。实测峰值显存 13.2 GB，每轮含验证约 20 秒。序列长度 200 时显存超过 24 GB，这是正式实验取 100 的原因。

2.2.7 与 HLLM 原版的差异

项目	HLLM	miniHLLM
Item LLM	TinyLlama 1B / Baichuan 7B，端到端微调	GPT-2 small，冻结，输出上接 1.2M 参数的适配器
User LLM	同上，端到端微调	GPT-2 small，全参数微调
物品文本	标题、描述、属性	标题与类型
物品读出	附加 [ITEM] token 的隐藏状态	掩码平均池化
训练目标	采样负例的 InfoNCE	全量 softmax 交叉熵
是否有 ID embedding	无	无（对照 H4 加回）
数据规模	亿级交互	100 万交互

保留的是 HLLM 的核心思想：物品表示完全来自文本、两层 LLM 分别负责物品与用户、输入输出共用物品表、对比式训练目标。简化的是两侧 LLM 的规模和物品侧的微调方式。

2.2.8 对照模型设计

为了把 “LLM 的作用” 拆解成可检验的部分，构造以下对照。所有模型共享数据划分、物品文本向量、打分函数、损失、训练预算与三个随机种子，每组对照只改变一个因素：

编号	模型	用户模型	物品表示	与 H1 的唯一差异	回答的问题
P	Popularity	无	无	–	底线
S1	SASRec (ID)	2 层 Transformer，从零训练，维度 64	可训练 ID 向量	骨干与物品表示都不同	序列建模的基础水平
S2	SASRec (text)	同上	仅文本向量	只换骨干	冷门桶弱势来自骨干还是表示
S3	SASRec (ID + text)	同上	ID + 文本相加	骨干不同、多了 ID	阶段一思路在序列模型上的效果
H1	**miniHLLM (text)**	预训练 GPT-2，微调	仅文本向量	–	主模型
H2	GPT-2 from scratch	同结构，随机初始化	仅文本向量	只换初始化	Q2：预训练的贡献
H3	miniHLLM, frozen	预训练 GPT-2，冻结	仅文本向量	只冻结 User LLM	Q3：微调程度
H4	miniHLLM (text + ID)	预训练 GPT-2，微调	文本 + ID	只加 ID 分支	Q1：ID 是否冗余

SASRec 骨干使用 `nn.TransformerEncoder`，2 层、2 头、前置 LayerNorm、GELU、可学习位置编码、dropout 0.1，是 Kang & McAuley (2018) 的标准配置。S2 是关键的桥梁：它与 H1 的物品表示、打分、损失完全一致，只有骨干不同，因此 S2 与 H1 的差异可以归因于 “预训练 GPT-2 对比从零训练的小 Transformer”，S1 与 S2 的差异可以归因于 “ID 对比文本”。

2.3 大语言模型的具体作用

综合两个阶段，LLM 在系统中承担三种角色，分别作用在数据流的不同位置：

角色	作用位置	阶段	不用 LLM 时的替代	对应的对照
物品侧语义编码器	$\mathbf{t}_i$：文本 $\rightarrow$ 768 维向量	一、二	可训练 ID 向量	S1 对 S2、S3
用户侧序列建模骨干	$\mathbf{h}_t$：向量序列 $\rightarrow$ 用户状态	二	从零训练的 2 层 Transformer	S2 对 H1
预训练知识的初始化	User LLM 的权重	二	同结构随机初始化	H2 对 H1

物品侧语义编码器。冻结 GPT-2 把标题与类型映射为向量，使标题相近、类型相同的电影在向量空间中靠近。这是整个系统中唯一不依赖交互数据的信息来源，也是物品冷启动能力的来源。阶段一把它作为 NCF 的附加输入；阶段二把它作为物品的全部表示。

用户侧序列建模骨干。预训练 GPT-2 的 12 层 Transformer 直接处理用户行为序列。它带来两样东西：87M 参数的容量，以及在文本上学到的注意力模式与层间表示。前者可以由任何大模型提供，后者只有预训练模型有。两者的贡献由 H2 对照分离。

预训练知识的迁移。GPT-2 的预训练语料与电影评分序列毫无关系，若 H1 优于 H2，说明语言模型学到的是通用的序列建模能力（如何根据前文聚合信息、如何建立长程依赖），而不是语言知识本身。这是 Q2 的实质。

阶段二没有使用 LLM 的生成能力：GPT-2 不生成推荐理由、不生成物品描述、不参与评测。本项目中 LLM 的作用限于表示学习与序列建模，生成式用法作为改进方向在第 4 节讨论。

2.4 实现要点

代码位于 `recommender/miniHLLM/`，不依赖阶段一的任何代码：

文件	内容	行数
`item_features.py`	冻结 GPT-2 提取电影文本向量	85
`seq_data.py`	按时间排序、leave-one-out 划分、右侧填充、历史掩码、热度统计	100
`models.py`	ItemEncoder、SASRecBackbone、GPT2Backbone、SeqRecommender、PopularityRecommender	150
`train.py`	训练、早停、全量排序评测、冷启动分桶、案例输出	220
`run_all.sh`、`run_extra.sh`	主实验与补充实验	
`summarize.py`、`plot_results.py`	汇总表与图	

实现过程中解决的两个影响结果正确性的问题：

- **特征各向异性**（2.2.3 节）。未做标准化时模型在前 10 轮几乎不学习；这是预训练语言模型句向量的已知问题，处理方式常规，但对本任务影响很大。
- **评测的数值精度**。打分矩阵形状为（用户数 × 3953），bf16 在大数值下分辨率不足，会把大量电影判成同分；排名若按“严格大于”统计会严重高估命中率（预实验中出现过验证 HR@10 虚高到 0.58 的情况）。为此打分在 `autocast` 内显式切回 fp32，并采用悲观排名：与目标同分的电影一律视为排在目标之前。若模型退化到给所有电影同分，该算法给出 0 而非虚高的命中率。

3 实验结果

3.1 实验设置

数据。 MovieLens-1M：6,040 位用户，3,883 部电影，1,000,209 条评分。所有评分视为交互正样本。每位用户至少 20 条交互，序列长度中位数 96，最长 2,314。

划分。阶段一沿用课程框架：固定种子 42 记录级随机打乱，8:1:1 划分，训练 800,167 条、验证与测试各 100,021 条。阶段二按用户时间序列

leave-one-out：每位用户最后一条交互为测试目标，倒数第二条为验证目标；验证时输入 $[i_1, \ldots, i_{n-2}]$，测试时输入 $[i_1, \ldots, i_{n-1}]$，均截断到最近 100 条。

阶段二超参数。

项目	SASRec 系列	GPT-2 系列
序列最大长度	100	100
物品向量维度	64	768
骨干	2 层 2 头，dropout 0.1	GPT-2 small 12 层 12 头
学习率	1e-3	1e-4（随机初始化另测 3e-4）
权重衰减	0.01	0.01
批大小	128 用户	128 用户
最大轮数	200（S1 延长到 400）	200
早停耐心	20	20
温度 τ 初始值	0.05，可学习	0.05，可学习
随机种子	42、2026、3407	42、2026、3407

序列长度的选择依据一组预实验：SASRec (ID) 在截断 50 与 200 下训练 60 轮，测试 HR@10 分别为 0.169 与 0.202，说明更长的上下文有明显收益；但 GPT-2 在长度 200 时显存超过 24 GB，折中取 100。学习率在 10 轮预实验中从 5e-5、1e-4、3e-4 中选取。S1 在 200 轮上限时三个种子的最佳轮次均为 197 到 200，未收敛，故延长到 400 轮并以该结果为准。

环境。单张 NVIDIA RTX 3090 (24 GB)，Python 3.12，PyTorch 2.2.1 + CUDA 12.1，Transformers 4.40.2。GPT-2 系列训练峰值显存 13.2 GB，每轮含验证约 20 秒；SASRec 系列峰值 1.2 GB，每轮不到 1 秒。

3.2 评价指标

对每位用户，模型对全部有效电影打分，屏蔽该用户历史中出现过的电影，取目标电影的排名 r_u（从 0 计，同分按悲观处理）。

$$\mathrm{HR@}K = \frac{1}{|U|}\sum_{u\in U}\mathbb{I}(r_u < K), \qquad \mathrm{NDCG@}K = \frac{1}{|U|}\sum_{u\in U}\frac{\mathbb{I}(r_u < K)}{\log_2(r_u + 2)}$$

leave-one-out 下每位用户只有一个目标，NDCG@K 退化为命中时的位置折损。HR@10 的含义是 “目标电影出现在前 10 的用户比例”，NDCG@10 进一步奖励靠前的位置。全量排序没有负采样，数值远低于采样 100 个负例的协议，但更接近真实部署。

冷启动分桶。按测试目标电影在训练数据中的出现次数，把 6,040 位用户分为三桶：少于 100 次（507 人）、100 到 500 次（2,195 人）、500 次以上（3,338 人），分别计算 NDCG@10。阶段一的评测使用课程框架原有逻辑，与阶段二不可直接比较。

3.3 阶段一结果

课程框架下，全量数据训练 3 轮，按验证 NDCG@10 选择最佳轮次：

数据集	HR@10	HR@20	NDCG@10	NDCG@20
验证集（最佳轮）	0.5374	0.6561	0.1297	0.1314
测试集	0.5252	0.6501	0.1253	0.1288

三轮训练中损失与验证指标逐轮改善，系统能为用户生成有效的 Top-10 推荐。但该协议下每位用户在测试集中平均有约 17 部目标电影，HR@10 的含义是 “前 10 中至少命中一部”，数值自然偏高；且没有基线对照，无法判断文本特征的贡献。这正是阶段二要解决的问题。

3.4 阶段二主结果

表 1 与表 2 为测试集整体指标，三个种子的均值 ± 样本标准差，Popularity 无随机性只运行一次。

表 1: 测试集整体指标（Top-10）

模型	可训练参数	最佳轮次	HR@10	NDCG@10
P Popularity	0	–	0.037	0.018
S1 SASRec (ID)	0.36M	303, 246, 279	0.289 ± 0.006	0.164 ± 0.002
S2 SASRec (text)	0.16M	115, 199, 192	0.292 ± 0.005	0.167 ± 0.002
S3 SASRec (ID + text)	0.42M	146, 127, 101	**0.297 ± 0.008**	0.170 ± 0.005
H2 GPT-2 scratch (lr 1e-4)	87.6M	16, 14, 16	0.283 ± 0.003	0.167 ± 0.003
H2′ GPT-2 scratch (lr 3e-4)	87.6M	9, 7, 8	0.281 ± 0.001	0.166 ± 0.001
H3 miniHLLM (frozen)	1.8M	54, 42, 53	0.283 ± 0.001	0.164 ± 0.002
H4 miniHLLM (text + ID)	90.7M	24, 25, 27	0.292 ± 0.006	0.172 ± 0.003
H1 **miniHLLM (text)**	87.6M	24, 25, 25	0.294 ± 0.002	**0.173 ± 0.001**

所有学习模型都远高于热门推荐底线（NDCG@10 高出 8 倍以上），说明在 “预测下一部” 任务上个性化建模是必要的。SASRec (ID) 的 HR@10

表 2: 测试集整体指标（Top-20）

模型	HR@20	NDCG@20
P Popularity	0.068	0.026
S1 SASRec (ID)	0.393 ± 0.006	0.190 ± 0.002
S2 SASRec (text)	0.398 ± 0.007	0.194 ± 0.001
S3 SASRec (ID + text)	0.397 ± 0.006	0.195 ± 0.005
H2 GPT-2 scratch (lr 1e-4)	0.375 ± 0.005	0.190 ± 0.003
H2′ GPT-2 scratch (lr 3e-4)	0.383 ± 0.005	0.191 ± 0.002
H3 miniHLLM (frozen)	0.386 ± 0.001	0.190 ± 0.002
H4 miniHLLM (text + ID)	**0.399 ± 0.003**	0.199 ± 0.002
H1 **miniHLLM (text)**	0.396 ± 0.002	**0.199 ± 0.001**

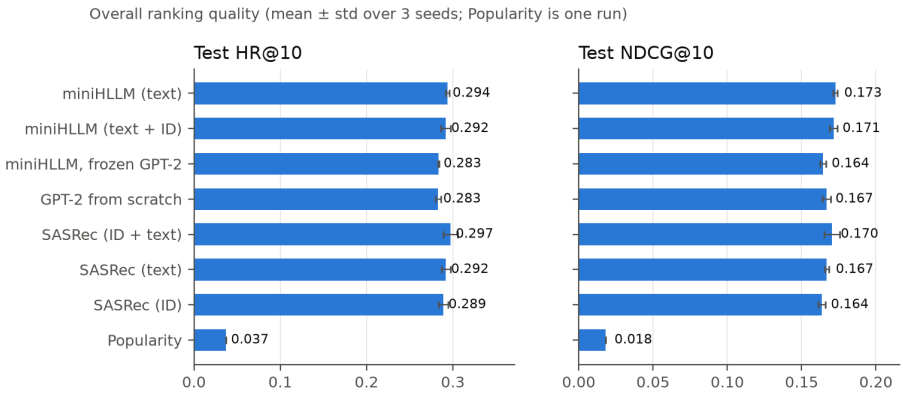


图 1: 测试集 HR@10 与 NDCG@10，误差线为三种子样本标准差。

为 0.289，与文献中 ML-1M 全量排序协议下的 SASRec 结果一致，表明实验管线可信。

miniHLLM (text) 取得最高的 NDCG@10 与 NDCG@20，HR@10 与 SASRec (ID + text) 在一个标准差内。两者参数量相差 200 倍，这一点在 3.6 节讨论。miniHLLM 三个种子的标准差为 0.001，是所有模型中最小的；SASRec 系列的标准差在 0.002 到 0.005 之间。

3.5 收敛速度

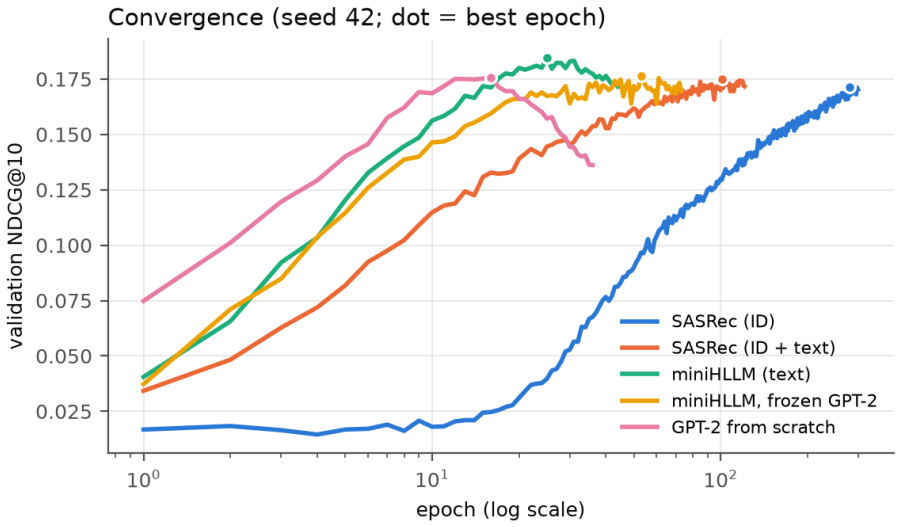


图 2: 种子 42 的验证集 NDCG@10 随训练轮次变化，横轴为对数刻度，圆点为早停选中的最佳轮。

收敛行为差异是本实验最显著的现象之一（图 2）。SASRec (ID) 前 15 轮几乎不动，ID 向量从随机初始化中建立结构需要时间，之后缓慢上升，最佳轮次在 250 到 300 轮之间。miniHLLM 第 1 轮就达到 SASRec (ID) 第 30 轮的水平，25 轮到达最佳，之后开始过拟合。随机初始化的 GPT-2 上升最快，但在 15 轮左右达到峰值后迅速过拟合，峰值低于预训练版本。冻结 GPT-2 的版本上升平缓，50 轮左右到达最佳，之后保持稳定不过拟合。

以墙钟时间计，miniHLLM 一次训练约 15 分钟（含早停等待的 20 轮），SASRec (ID) 约 3.5 分钟。单轮代价高 15 倍，但所需轮次少 10 倍。

3.6 冷启动分析

表 3: 按目标电影训练热度分桶的测试 NDCG@10

模型	<100 (n=507)	100–500 (n=2,195)	≥ 500 (n=3,338)
P Popularity	0.000	0.000	0.033
S1 SASRec (ID)	0.052 ± 0.004	0.119 ± 0.005	0.210 ± 0.003
S2 SASRec (text)	0.042 ± 0.004	0.118 ± 0.004	0.218 ± 0.004
S3 SASRec (ID + text)	0.063 ± 0.005	0.123 ± 0.006	0.218 ± 0.006
H2 GPT-2 scratch	0.043 ± 0.004	0.113 ± 0.002	0.221 ± 0.005
H3 miniHLLM (frozen)	**0.065 ± 0.006**	**0.124 ± 0.004**	0.206 ± 0.005
H4 miniHLLM (text + ID)	0.046 ± 0.005	0.122 ± 0.003	0.223 ± 0.007
H1 miniHLLM (text)	0.043 ± 0.003	0.119 ± 0.004	**0.228 ± 0.005**

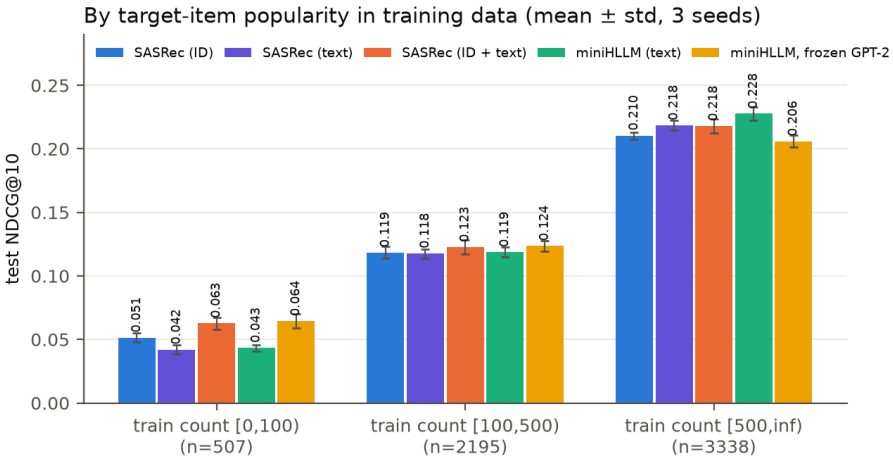


图 3: 各热度桶的测试 NDCG@10。

热门推荐在前两桶的 NDCG 为 0，说明冷门目标只能靠个性化模型命中。三个值得注意的现象：

1. **miniHLLM 的整体优势全部来自热门桶**。它在 ≥ 500 桶的 NDCG@10 为 0.228，比 SASRec (ID) 高 9%；但在 <100 桶只有 0.043，低于 SASRec (ID) 的 0.052。全参数微调的 87M 参数把容量用在了交互充分的热门电影上。

2. **冻结 GPT-2 把这个权衡反了过来**。只训练 1.8M 参数适配器的 H3 在冷门桶达到 0.065，是所有模型最高，热门桶则最低。整体指标上 H3

与 SASRec (ID) 持平，但两者的强项分布完全不同：H3 的能力来自文本语义，不依赖交互次数；SASRec (ID) 的能力随交互次数增长。

3. **文本的冷启动收益需要与 ID 配合**。在 SASRec 骨干下，仅文本的 S2 在冷门桶（0.042）并不比纯 ID 的 S1（0.052）好，两者相加的 S3 才达到 0.063。这与直觉相反：一种解释是本实验的 "冷门" 桶仍有 20 到 100 次交互，ID 向量足以学到有用信息；而仅文本的模型为所有电影共享一个适配器，冷门电影若与某热门电影标题或类型相近，会被拉向后者。S2 与 H1 在冷门桶的表现几乎相同（0.042 与 0.043），说明这个弱点来自 "仅文本" 的物品表示，而不是 GPT-2 骨干。

3.7 消融分析：回答三个研究问题

Q1：文本表示能否替代 ID？能。H1 不含 ID，整体 NDCG@10 高于所有含 ID 的 SASRec 变体；H4 在 H1 基础上加回 ID，整体指标没有变化（0.172 对 0.173），冷门桶也只从 0.043 升到 0.046。对 GPT-2 骨干而言，文本表示已经足够，ID 是冗余的。这是 HLLM"无 ID" 设计在小数据集上的验证。

Q2：语言预训练能否迁移？能，且效果稳定。H1 与 H2 结构完全相同，只差初始化：预训练的 NDCG@10 为 0.173 ± 0.001，随机初始化在两个学习率下分别为 0.167 ± 0.003 和 0.166 ± 0.001，差距是种子标准差的 3 到 6 倍。随机初始化模型在 15 轮内即过拟合，预训练模型可以训到 25 轮，说明预训练权重起到了正则化作用。考虑到 GPT-2 的预训练语料与电影评分序列毫无关系，这一迁移效果表明语言模型学到的通用序列建模能力（而非语言知识本身）对行为序列有用。

Q3：微调程度与冷启动的关系？微调程度是热门与冷门之间的旋钮。冻结（H3, 1.8M 可训练参数）冷门最好、热门最差；全参数微调（H1, 87.6M）热门最好、冷门最差；两者整体指标相差 0.9 个点。在实际系统中，可根据业务对长尾的重视程度选择中间方案（如 LoRA），本项目未做这一组实验。

3.8 推荐案例

用户 1 的测试序列最后五部为《虫虫危机》《蚁哥正传》《钟楼怪人》《大力士》《花木兰》，真实的下一部是《风中奇缘》。两个模型的 Top-10 推荐（* 为命中）：

排名	miniHLLM (text)	SASRec (ID)
1	Lion King, The (1994)	Prince of Egypt, The (1998)
2	Prince of Egypt, The (1998)	Lion King, The (1994)
3	Anastasia (1997)	Anastasia (1997)
4	*** Pocahontas (1995)**	*** Pocahontas (1995)**
5	Little Mermaid, The (1989)	South Park: Bigger, Longer and Uncut (1999)
6	Jungle Book, The (1967)	Alice in Wonderland (1951)
7	Alice in Wonderland (1951)	Beavis and Butt-head Do America (1996)
8	Robin Hood (1973)	Stuart Little (1999)
9	Balto (1995)	Sword in the Stone, The (1963)
10	Lady and the Tramp (1955)	Fantasia 2000 (1999)

两个模型都在第 4 位命中。差异在未命中的部分：miniHLLM 的 10 部全是迪士尼风格的动画片，与用户近期观看高度一致；SASRec (ID) 混入了《南方公园》《瘪四与大头蛋》两部成人向动画，它们与儿童动画在 ID 共现统计上相近（同为 “动画” 类型的热门片），但语义上差别很大。这个例子直观展示了文本语义对推荐列表一致性的作用。

4 项目总结

4.1 主要成果

1. **完成了两代 LLM 增强推荐系统**。阶段一实现课程框架的 GPT-2 + NCF；阶段二设计并实现 miniHLLM，把 HLLM 的分层 LLM 结构适配到 GPT-2 small 与单卡环境，代码约 550 行，训练一次 15 分钟。

2. **建立了严格受控的对照实验**。8 个模型、3 个种子、统一的时间 leave-one-out 协议与全量排序评测，加上按热度分桶的冷启动分析，共 28 次训练，全部日志、逐轮曲线与指标归档在 `recommender/miniHLLM/results/main/`。

3. **得到了可复现的结论**。不含 ID 的 miniHLLM 整体 NDCG@10 最高（0.173）、方差最小、收敛最快；语言预训练带来 0.6 个点的稳定增益；ID 对 miniHLLM 冗余；微调程度决定模型偏向热门还是冷门。

4. **识别并解决了两个影响结果正确性的实现问题**：GPT-2 特征的各向异性，以及 bf16 评测中平局导致的指标虚高。

4.2 不足

- **物品文本信息量小**。只有标题和类型，平均十几个 token。文本分支的上限受此限制，这也是仅文本模型在冷门桶不占优的可能原因之一。
- **Item LLM 没有微调**。 HLLM 的核心是两个 LLM 端到端训练，本项目受显存限制只在冻结特征上训练适配器。物品侧与用户侧的联合优化没有验证。
- **参数量不对等**。miniHLLM 的 87M 参数与 SASRec 的 0.4M 相差 200 倍，H1 对 S3 的 0.3 个点优势不能单独归因于预训练或结构，H2 对照只隔离了初始化。一个参数量接近 GPT-2 的从零训练 SASRec 会使比较更公平。
- **数据集单一且偏小**。 ML-1M 的 3,883 部电影中交互少于 100 次的目标只有 507 个，冷启动桶的统计功效有限；结论能否推广到更大、更稀疏的数据集未知。
- **微调程度只有冻结与全参数两个极端**。 LoRA、仅微调顶层等中间方案未测。

4.3 改进方向

1. **用生成式 LLM 丰富物品文本**。用本地指令模型为每部电影生成剧情简介、风格与受众描述，再编码，预计直接提升文本分支尤其是冷门桶的表现。
2. **Item LLM 的参数高效微调**。对物品侧 GPT-2 加 LoRA，每步只对 batch 内出现的电影重新编码，其余使用缓存向量（记忆库思路），使两侧 LLM 联合训练在单卡上可行。
3. **微调程度的连续扫描**。在冻结与全参数之间加入 LoRA 秩 8、16、64 和仅微调顶部 k 层，绘制整体指标与冷门桶指标随可训练参数量的变化曲线，把 Q3 的 “旋钮” 定量化。
4. **等参数量对照与更大数据集**。增加与 GPT-2 同规模的从零训练 Transformer，并在 Amazon Reviews 等物品更多、更稀疏的数据集上验证。

5. **生成式推荐范式的对比**。在相同物品文本向量上训练 RQ-VAE 得到语义 ID，实现 TIGER 风格的生成式检索，与 miniHLLM 的稠密检索做范式级对比。

附录：复现

```
git lfs pull                        # 数据、GPT-2 权重
cd recommender/miniHLLM
python item_features.py             # 电影文本向量 -> weight/item_text_features.pt
./run_all.sh && ./run_extra.sh      # 28 次训练 -> results/main/
python summarize.py                 # 汇总表
python plot_results.py              # 图 -> figures/
```

阶段一：

```
cd recommender/base_LLM_NCF
python gpt2.py && python main.py --epochs 3
```

参考文献

- Chen, J. et al. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. arXiv:2409.12740, 2024.
- Kang, W.-C. & McAuley, J. Self-Attentive Sequential Recommendation. ICDM 2018.
- He, X. et al. Neural Collaborative Filtering. WWW 2017.
- Rendle, S. et al. BPR: Bayesian Personalized Ranking from Implicit Feedback. UAI 2009.
- Radford, A. et al. Language Models are Unsupervised Multitask Learners. OpenAI, 2019.
- Ethayarajh, K. How Contextual are Contextualized Word Representations? EMNLP 2019.
- Timkey, W. & van Schijndel, M. All Bark and No Bite: Rogue Dimensions in Transformer Language Models Obscure Representational Quality. EMNLP 2021.

- Petrov, A. & Macdonald, C. gSASRec: Reducing Overconfidence in Sequential Recommendation Trained with Negative Sampling. RecSys 2023.
- Rajput, S. et al. Recommender Systems with Generative Retrieval. NeurIPS 2023.